

Betriebssysteme

Interrupts und System Calls

Andreas Scheibenpflug, MSc
SE Bachelor (BB/VZ), 2. Semester

Die HW sagt ‚Hallo‘

- Konzeptionell ist ein Interrupt (Unterbrechung) das Aufzeigen eines Ereignisses in einer HW Komponente
- Dieses Ereignis wird von der Hardware an die CPU geschickt
 - Wird als Unterbrechungsanforderung bzw. Interrupt Request (IRQ) bezeichnet
- Die CPU/das Betriebssystem sollte dann mit einer Aktion darauf reagieren
 - Diese Aktion wird als Unterbrechungsroutine bzw. Interrupt Handler bezeichnet
 - Der Interrupt Handler wird vom OS/Treibern zur Verfügung gestellt
- Vorteil: Keine verschwendete CPU Zeit durch Polling der HW
 - Polling: Periodische Abfrage der HW nach Änderungen

Beispiel – Ablauf Interrupt

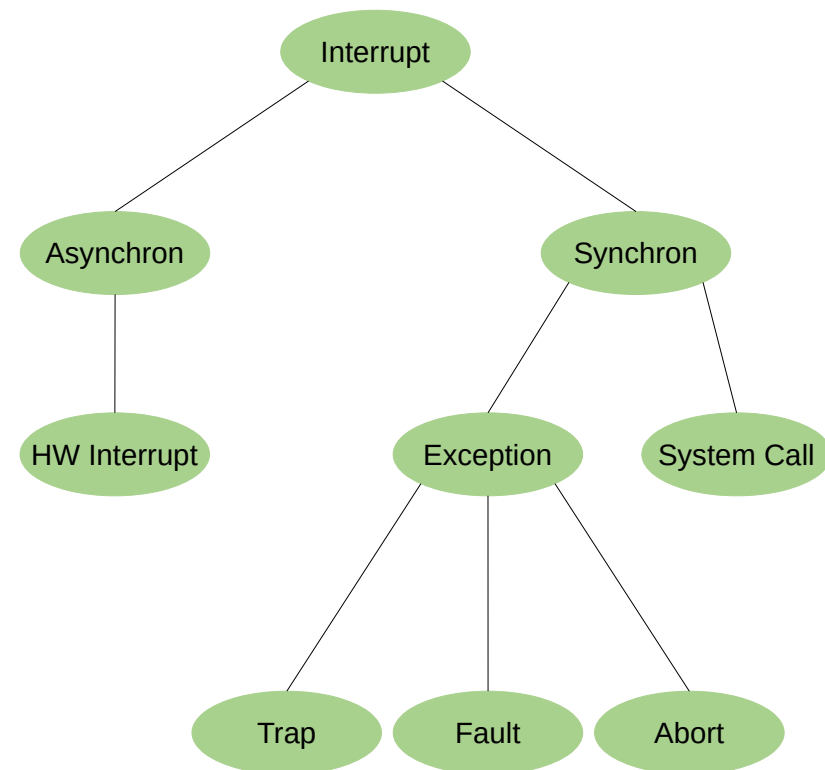
- Benutzer*in bewegt die Maus
- Die Maus erkennt die Bewegung und schickt die Daten an die CPU, z.B.
 - in welche Richtung wurde die Maus wie viel bewegt
 - welche Tasten wurden gedrückt
- Die CPU erkennt den Interrupt und unterbricht die Ausführung des aktuellen Prozesses
- Auf den Interrupt wird mit einem Interrupt Handler des Maus Treibers reagiert
- Interrupt Handler verarbeitet die Daten und führt weitere Aktionen durch (z.B. werden Prozesse (GUI) über die aktualisierten Daten informiert)
- Nach der Ausführung des Interrupt Handlers wird der unterbrochene Prozess fortgesetzt

Quellen von Interrupts

- Neben der Quelle HW gibt es noch andere Möglichkeiten wie ein Interrupt ausgelöst werden kann
- Prozessor: Wird durch die CPU bei der Ausführung eines Programms ausgelöst
 - z.B. Division durch Null, Adressfehler (Segfaults), ...
- Software: Ein Programm löst einen Interrupt aus, um einen Service des OS zu verwenden
 - Damit wird in den Kernel Mode gewechselt
 - Das OS führt die Aufgabe aus und returniert zum aufrufenden Programm
 - Wird als System Call bezeichnet (z.B. Einlesen einer Datei, Laden von Daten aus dem Internet, Verwenden einer Semaphore oder eines Mutex,...)

Kategorisierung (~)

- **Asynchron**
 - Tritt parallel zur Ausführung der aktuellen Instruktion auf
- **Trap**
 - Instruktion löst Trap aus
 - Ausführung wird unterbrochen
 - Interrupt wird vom OS behandelt
 - Ausführung wird bei der nachfolgenden Instruktion fortgesetzt
 - Beispiel: Breakpoint, System Call,...
- **Fault**
 - Eine Exception die behandelt werden kann
 - Instruktion löst Fault aus
 - Ausführung wird unterbrochen
 - Interrupt wird vom OS behandelt
 - Instruktion die den Fault ausgelöst hat, wird wiederholt
 - Beispiel: Div. durch Null, Page Fault (siehe Foliensatz zu Speicherverwaltung)
- **Abort**
 - Es liegt ein gravierender Fehler vor
 - Die Ausführung der Anwendung wird abgebrochen
 - Beispiel: Hardware Fehler, Illegale/Inkonsistente Werte in System Tabellen



Ablauf Interrupt - HW

- Geräte sind über einen Bus mit dem (Programmable) Interrupt Controller (PIC) verbunden
- Der Interrupt Controller agiert als Multiplexer, priorisiert die eingehenden Interrupt Requests (IRQs) und setzt ein Bit (IEB) um der CPU zu signalisieren, dass ein Interrupt anliegt
- CPU überprüft nach jeder ausgeführten Instruktion dieses Bit. Ist es gesetzt dann
 - Sichert die CPU den Zustand des aktuellen Prozesses (Instruction Pointer, Register, Stack,...mehr dazu im Foliensatz Prozesse)
 - Signalisiert die CPU dem PIC dass sie bereit ist und erhält die Nummer des Interrupts vom PIC
 - Die Nummer ist einem Interrupt Handler zugeordnet
 - Der Interrupt Handler wird ausgeführt
 - Nach Beendigung des Interrupt Handlers wird der zuvor gestoppte Prozess wieder geladen und weiter ausgeführt

Interrupt Handler

- Werden von Treibern/OS zur Verfügung gestellt
- Werden vom OS für Interrupts registriert
- Beispiel Linux: Interrupt Descriptor Table (IDT)
 - Tabelle im Hauptspeicher die von Linux verwaltet wird
 - Enthält das Mapping Interrupt (Nummer) → Interrupt Handler
 - Wird zur Boot Zeit befüllt
 - Dem Prozessor wird über ein Register mitgeteilt, wo sich diese Tabelle im Hauptspeicher befindet
 - Die CPU verwendet diese Tabelle um Interrupt Handler zu laden
 - Mehrstufige Implementierung
 - Interrupt Handler sollen möglichst schnell sein, da Interrupts blockiert werden
 - Top Half: Interrupt Handling (Interrupts blockiert)
 - Es werden z.B. die Daten von der HW (Netzwerkpakete, Datenblöcke,...) gelesen und in einen Puffer gespeichert
 - Bottom Half: Weiterführende Aufgaben (Interrupts wieder aktiviert)
 - Können asynchron ausgeführt werden
 - Verarbeitung der gelesenen Daten, Weiterleitung der Daten an darüber-liegende Schichten (Protokollschicht, Dateisystem,...)

```
/*
 * Linux IRQ vector layout.
 *
 * There are 256 IDT entries (per CPU - each entry is 8 bytes) which can
 * be defined by Linux. They are used as a jump table by the CPU when a
 * given vector is triggered - by a CPU-external, CPU-internal or
 * software-triggered event.
 *
 * Linux sets the kernel code address each entry jumps to early during
 * bootup, and never changes them. This is the general layout of the
 * IDT entries:
 *
 * Vectors  0 ... 31 : system traps and exceptions - hardcoded events
 * Vectors 32 ... 127 : device interrupts
 * Vector 128       : legacy int80 syscall interface
 * Vectors 129 ... LOCAL_TIMER_VECTOR-1
 * Vectors LOCAL_TIMER_VECTOR ... 255 : special interrupts
 */
```

Beispiel Timer Interrupt

- Programmable Interval Timer (PIT)
 - PIT ist im Mainboard des Computers untergebracht
 - Er erzeugt Interrupts in periodischen Intervallen (n mal pro Sekunde)
 - Liefert Ticks: Zahl der gesendeten Timer-Interrupts pro Sekunde, meistens 50, 60 oder 100 Ticks → ca. alle 10 ms ein Interrupt
 - Intervall-Länge ist programmierbar
 - Timer-Interrupt läuft mit hoher Priorität
- Interrupt Handler für den Timer Interrupt
 - Aktualisiert die Uptime (Zeit, die seit dem Systemstart vergangen ist) alle 10 ms (= 1 Tick)
 - Aktualisiert die Systemzeit
 - Überprüft die Zeitscheibe des aktuellen Prozesses – falls abgelaufen, wird der Scheduler gestartet, der die CPU einem anderen Prozess zuteilt (mehr dazu im Foliensatz Prozesse)
 - Aktualisiert Ressourcen-Nutzungsstatistik

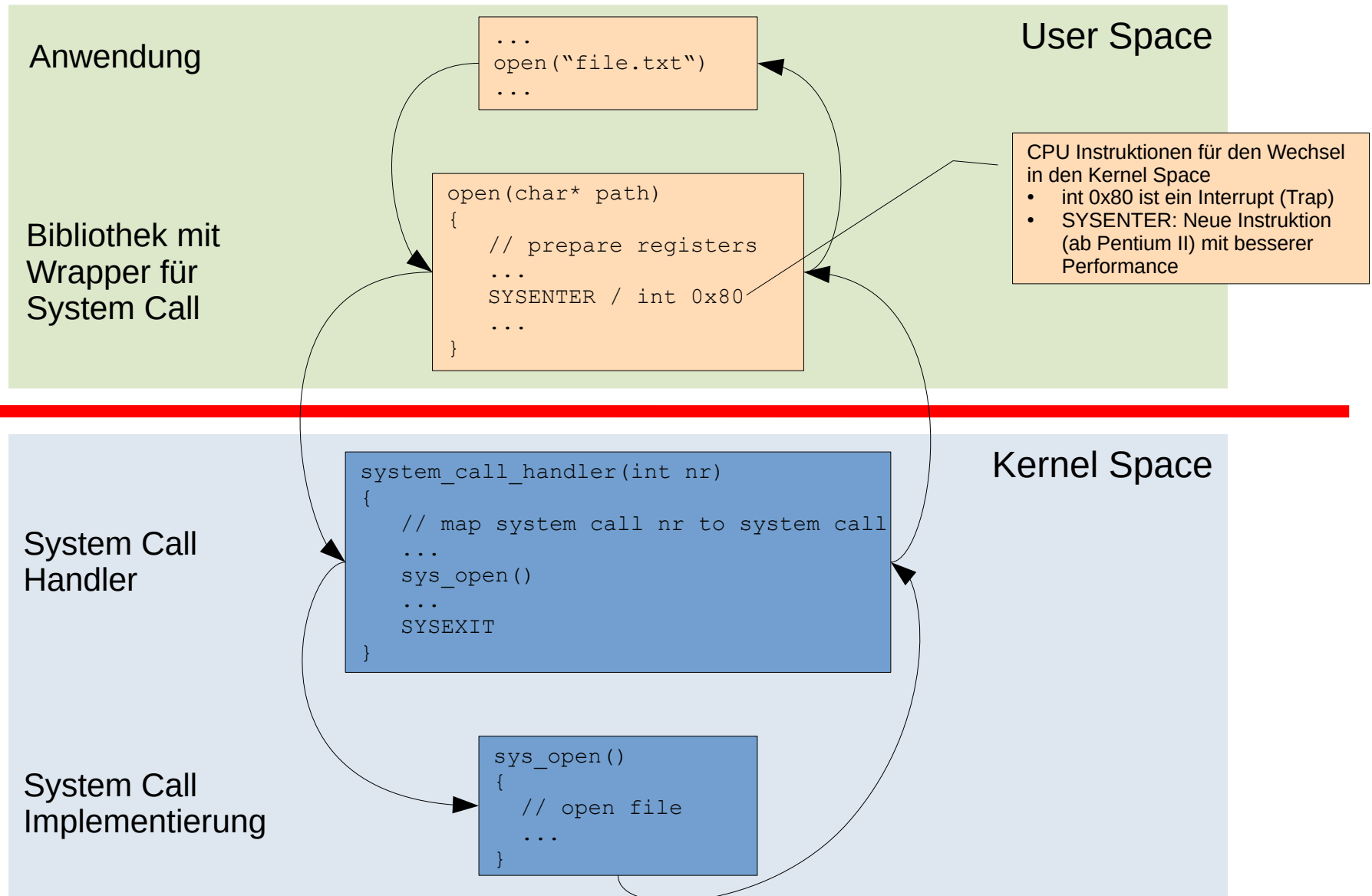
System Calls

- Ein OS stellt dem/der Anwendungsentwickler*in eine Reihe von Funktionalitäten zur Verfügung
 - System Calls erlauben diese Funktionalitäten einfach in Anwendungen zu verwenden
 - Abstrahieren die Komplexität der darunter liegenden HW
 - Ermöglichen Aktionen, die für Anwendungsprogramme nicht erlaubt sind (z.B. direkter Zugriff auf die HW)
- Beispiele
 - Abstrahierte Formen der Ein- und Ausgabe auf Geräte, z.B.
 - Lesen/Schreiben auf der Festplatte
 - Senden/Empfangen von Paketen über die Netzwerkschnittstelle
 - Interaktion mit Prozessen, dem Arbeitsspeicher,... z.B.
 - Starten von neuen Prozessen
 - Synchronisieren/Interaktion mit anderen Prozessen
 - Anfordern/Freigeben von Arbeitsspeicher

System Calls

- Im Gegensatz zu Prozessen von Benutzer*innen läuft das OS in einem privilegierten Modus
 - Prozesse im User Mode (Space) haben eingeschränkte Rechte um Fehler zu verhindern, die andere Programme oder das OS negativ beeinflussen könnten
 - Das OS läuft im Kernel Mode (Space) und hat den kompletten CPU Befehlssatz, Arbeitsspeicher und Zugriff auf die HW zur Verfügung
 - Diese unterschiedlichen Modi sind CPU-unterstützt, z.B. bei Intel
 - Ring 0 für Kernel Mode
 - Ring 3 für User Mode
- Dieser Übergang von User Space in Kernel Space wird mit einer speziellen CPU Instruktion angestoßen, welche einen Interrupt (Trap) auslöst
 - Der aufrufende Prozess wird dabei unterbrochen, bis der Kernel die Bearbeitung des System Calls beendet hat und das Ergebnis zurückliefert
 - Der Aufruf eines System Calls funktioniert semantisch wie ein normaler Funktionsaufruf

System Call - Ablauf



System Calls in Linux

- Linux bietet ~ 500 System Calls an
 - Manche System Calls werden nicht auf allen Plattformen unterstützt
- System Calls werden im Normalfall nicht direkt aufgerufen
 - Werden über C Bibliotheken zur Verfügung gestellt (z.B. glibc)
 - Diese Funktionen sind meist „dünne Wrapper“ für den tatsächlichen System Call und machen oft nicht mehr als
 - Funktionsargumente und die Nummer des System Calls in die richtigen Register kopieren (wie vom Kernel erwartet)
 - In den Kernel Mode wechseln (den Trap ausführen)
 - Abholen des Return Codes und Rückgabe an den Aufrufer
 - Meist sind die Namen von System Calls und Wrapper ident (z.B. fork(..), open(..), chdir(..), poll(..),...)

Bsp. Mouse IRQ + System Calls

- Maus Treiber erhält IRQs über den PIC von der Maus
- Der Treiber schreibt die Daten des IRQs in eine Datei in /dev
- Verwenden der System Calls `open(..)` und `read(..)` zum Auslesen und Anzeigen der Daten im User Space

```
x=2, y=1, left=0, middle=0, right=0
x=4, y=3, left=0, middle=0, right=0
x=3, y=2, left=0, middle=0, right=0
x=3, y=3, left=0, middle=0, right=0
x=2, y=1, left=0, middle=0, right=0
x=3, y=2, left=0, middle=0, right=0
x=2, y=2, left=0, middle=0, right=0
x=2, y=1, left=0, middle=0, right=0
x=4, y=1, left=0, middle=0, right=0
x=1, y=0, left=0, middle=0, right=0
```

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

// gcc mouse.c -o mouse
// sudo ./mouse
int main(int argc, char** argv)
{
    int fd, bytes;
    unsigned char data[3];
    const char *device_path = "/dev/input/mouse1";
    int left, middle, right;
    signed char x, y;

    // open device file for mouse
    fd = open(device_path, O_RDONLY);
    if (fd == -1)
    {
        printf("ERROR Opening %s\n", device_path);
        return -1;
    }

    while(1)
    {
        // read blocks until new data is available and can be read
        bytes = read(fd, data, sizeof(data));

        if (bytes > 0)
        {
            left = data[0] & 0x1; // 00000001
            right = data[0] & 0x2; // 00000010
            middle = data[0] & 0x4; // 00000100

            x = data[1];
            y = data[2];
            printf("x=%d, y=%d, left=%d, middle=%d, right=%d\n",
                x, y, left, middle, right);
        }
    }
    return 0;
}
```

<https://stackoverflow.com/a/23317086>

Zusammenfassung

- Interrupts
- Ablauf eines Interrupts
- Interrupts aus Sicht des OS
- Funktionsweise System Calls